

AgentProf: Semantic Profiling for AI Agents

Anonymous submission

Abstract

AI agents increasingly orchestrate long-running, multi-step activities involving thousands to millions of interactions with users, tools, and system resources. To improve agent quality, safety, and cost efficiency, developers need to determine where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget. Traditional profiling answers analogous questions in systems software by aggregating resource consumption under responsible code paths rather than debugging one execution at a time. Yet existing agent observability tools are mainly built for debugging and tracing, not profiling. Agent behavior also requires a different profiling index: responsible entities are semantic categories such as task intent and workflow phase, while natural-language activities lack the stable semantic identifiers and cross-run attribution hierarchy provided by a runtime call stack. We propose a semantic operation stack model that adapts profiling to agent trajectories. Uniform `operation` records represent agent activities and effects, and query-time `operation stack` paths provide hierarchical attribution at different semantic granularities from the same data. `AGENT-PROF` implements this model as an offline Rust profiler with pluggable intent attribution and stack construction and emits pprof-compatible profiles. Our four RQs evaluate resource attribution, correspondence to real problems, tag accuracy, and profiling cost on real trajectories and public agent datasets.

Introduction

AI agents (Yang et al. 2024; Anthropic 2025; OpenAI 2025; Xie et al. 2024) orchestrate multi-step activities that span high-level intent—prompts, large language model calls, and tool invocations—and low-level system effects such as process spawns and file or network I/O. As agents evolve from short, scripted workflows into complex systems that run for hours or days, production teams accumulate many such trajectories over weeks or months.

To improve agent quality, safety, and cost efficiency, developers need to analyze operational behavior across trajectories. Which task categories consume the most token budget? Where do failures concentrate across workflows? Which

behavioral patterns trigger unsafe system effects? Answering these questions requires analysis across many trajectories (Mazaheri and Mazaheri 2026; Lù et al. 2025), which becomes costly for manual inspection or per-trajectory LLM evaluation (Zheng et al. 2023). In traditional systems software, profiling answers such questions by aggregating resource consumption under responsible entities, rather than debugging or tracing a single execution.

Yet existing agent tools support debugging and tracing rather than profiling (Section). Span-based tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) organize events into per-execution trees that are effective for diagnosing one run but do not aggregate resource consumption by semantic category across runs. Other systems (Datadog 2025; Laminar 2025) cluster inputs or extract structured signals, but input distributions do not by themselves attribute downstream model, tool, and system costs to the responsible work.

Adapting profiling to agent trajectories is challenging. In traditional software, stable function names identify responsible entities and runtime call nesting supplies the attribution hierarchy. Agent behavior differs: the responsible entities are semantic categories such as task intent and workflow phase, while two natural-language activities with the same intent need not share an identifier. Agent traces may preserve execution spans, but execution location alone does not define the semantic hierarchy needed to aggregate recurring work across runs.

Despite these differences, the fundamental profiling methodology transfers: project resources onto responsible entities, assign stable tags, and attribute them hierarchically. Agent observability needs profiling, not only debugging. We propose a semantic operation stack model with two components. Uniform `operation` records represent prompts, model calls, tool invocations, and system effects with additive measures. `operation stack` paths replace the fixed runtime call stack for semantic attribution: choosing task, phase, action, session, or effect fields changes the granularity while retaining the same recorded evidence.

We implement this model in `AGENTPROF`, an offline Rust profiler that compiles local agent trajectories and trace containers into pprof-compatible profiles (Figure 1). Pluggable intent-attribution methods derive stable tags from natural-language fields, and pluggable stack construction builds the

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

semantic attribution hierarchy. We evaluate whether the resulting profiles improve resource attribution, locate real problems, derive accurate tags, and remain practical on real trajectories and public agent datasets.

This paper makes three contributions:

1. **Semantic operation stack model.** We identify the missing profiling layer in agent observability and propose uniform operations plus query-time operation stacks for semantic resource attribution (Background and Motivation; Design).
2. **System: AGENTPROF.** We implement the model in a profiler with pluggable intent attribution and stack construction that produces pprof-compatible output (Design; Implementation).
3. **Evaluation.** We evaluate profiler output through resource attribution, correspondence to real problems, tag accuracy, and profiling cost on real trajectories and public datasets (Evaluation).

Background and Motivation

LLMs and AI Agents

We use *trajectory* for one recorded agent execution and *cross-run* for aggregation, transfer, or models learned across multiple trajectories. Session and span denote source-specific fields within one trajectory.

A typical agent trajectory interleaves two layers of activity. At the intent layer, the agent receives a user prompt, issues LLM calls, and invokes tools (code execution, web search, file editing). Tool invocations can trigger lower-layer system effects, including process spawns, file reads and writes, and network requests (Zheng et al. 2025). These layers repeat through a trajectory, and a corpus accumulates many trajectories over time.

System Profiling

Profiling aggregates resource consumption by responsible entity, unlike single-execution debugging and tracing. Traditional profilers sample execution state, attach each sample to its call stack, and merge identical stacks. Flame graphs (Gregg 2011) and pprof (Google 2024b) call graphs visualize these aggregates, while Perfetto SQL (Google 2024a) and pprof’s `tagroot/tagleaf` regroup supplied semantic attributes.

Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) record agent events as per-execution span trees, effective for debugging a single trajectory. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Adapting profiling to agent trajectories faces two core challenges. In traditional software, stable function identities and the call stack jointly identify which code path owns a sample at every level. First, agent profiling instead asks which semantic role, such as task intent or workflow phase, owns an

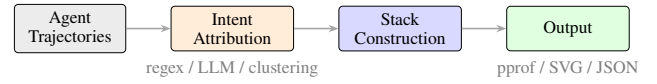


Figure 1: AgentProf pipeline. Agent trajectories pass through pluggable intent attribution and stack construction before standard profiler output.

effect across trajectories. Equivalent natural-language activities need not share a stable identifier. Second, when present, a trajectory’s emitted span tree encodes execution structure rather than cross-run semantic responsibility. The tree therefore cannot by itself determine whether a task, phase, action, or system effect should define the attribution granularity. An agent profiler must therefore derive stable semantic tags and construct an attribution hierarchy while retaining the original execution evidence.

Design Requirements for Agent Profiling

Traditional profiling obtains three mechanisms from the call stack. Agent profiling needs the same capabilities for agent trajectories:

- R1: Cross-layer resource projection.** Connect model, tool, and system resource consumption to the intent-layer semantic categories responsible for that work.
- R2: Stable tags for folding.** Derive short, stable, repeatable tags from natural-language activity so equivalent work can fold across trajectories.
- R3: Hierarchical attribution.** Construct semantic attribution hierarchies at the granularity required by the analysis rather than assuming one fixed execution tree answers every cross-run question.

Design

The three requirements—cross-layer resource projection, stable tags, and hierarchical attribution—drive the design. Operations (Section) represent intent and system-effect activities in one record and allow intent-layer tags to propagate to the effects they trigger. Intent attribution (Section) derives stable tags from natural-language fields. Operation stacks (Section) replace the fixed runtime call stack with query-time field projections that attribute resources at multiple semantic granularities from the same data.

The profiling pipeline has four stages: (1) parse raw trajectories into operations, (2) enrich operations through intent attribution or mapping rules, (3) project each operation onto a stack-frame sequence selected at query time, and (4) fold operations with identical stacks while summing their weights.

Semantic Operation Stack Model

Because agent activities span both the intent and system-effect layers described above, a profiler should represent both uniformly. AGENTPROF therefore represents every activity as an *operation*, a weighted record with string fields and additive measures. Illustrative string fields include `project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`,

domain, and status. A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event share this schema and differ only in populated fields.

The reported built-in profiles use token count, elapsed time to the next recorded event, and event count. The schema accepts arbitrary imported fields, so source operation IDs and native run, trace, span, tool, or parent paths remain available for source-native projections when the input supplies them.

Operation Stacks

In CPU profiling, the call stack determines which code paths share responsibility for a resource cost. An operation stack serves the same role for agent trajectories: it is an ordered list of fields $[f_1, f_2, \dots, f_k]$ that projects each operation o to the frame sequence $\langle o.f_1, o.f_2, \dots, o.f_k \rangle$. Operations with identical sequences are merged and their additive weights are summed.

Unlike a runtime call stack, these fields are chosen at query time. The same operations can therefore be attributed as a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a source-native trajectory view. Changing fields changes attribution without changing the recorded data. Sessions and spans are optional operation fields, so including session yields a debugging view while excluding it yields an aggregate profiling view.

A profile view is a triple (φ, σ, w) : predicate φ selects operations, stack function $\sigma = [f_1, \dots, f_k]$ maps an operation to its frame sequence, and weight function w assigns a non-negative measure. Built-in views cover token count, elapsed time, file operations, and network operations. This compact model lets the same evidence answer different attribution questions without adding another scientific abstraction.

Algorithms

Intent attribution. Intent attribution maps natural-language prompts to short, stable, repeatable tags, giving agent trajectories the identifiers that traditional profiling gets from function names. The framework supports three pluggable backends. Ordered regular-expression rules are the production default. A cached local LLM tagger can generate constrained short labels, and TF-IDF/K-Means clustering can discover recurring prompt groups to help author rules. For structured trajectories, mapping rules derive semantic fields from existing action and state fields.

Stack construction. Stack construction builds an attribution hierarchy from available fields. When users supply an ordered field list, projection is direct. When no field list is supplied, AGENTPROF can induce stacks by scoring adjacent operations for shifts in prompt content and structured fields, then placing boundaries between work phases. A configurable depth cap controls granularity. Both modes produce the same weighted stack representation and standard profiler outputs.

Implementation

AGENTPROF is an offline, post-hoc profiler implemented as a Rust CLI (Figure 1). It reads Codex (OpenAI 2025) and Claude Code (Anthropic 2025) local histories,

explicit operation JSONL, portable agent-session traces, and Chrome/Perfetto trace containers. AgentSight recordings (Zheng et al. 2025) enter after conversion to a supported operation or trace input. These importers reconstruct prompts, model calls, tool invocations, and recorded file, process, and network effects as uniform operations.

The implementation provides pluggable intent-attribution and stack-construction algorithms over one folding pipeline. Output formats include pprof protobuf, folded-stack text, standalone SVG flame graphs, and JSON summaries. Pprof profiles open with `go tool pprof -http`, and folded stacks render with `flamegraph.pl`.

Built-in profiles weight model calls by token count, timed operations by elapsed duration, and file or network operations by event count. All admitted results use positive integer weights. The current importer normalizes an imported zero value to one, so the paper makes no claim about faithful zero-valued measures until that behavior is corrected.

Evaluation

We ask four paper-level research questions. **RQ1: Does Semantic Profiling Improve Resource Attribution? RQ2: Does Profiler Output Correspond to Real Problems? RQ3: How Accurate Are the Tags? RQ4: What Is the Profiling Cost?** Together they test whether semantic profiling attributes real additive measures, brings real annotated problems to the analyst’s attention, retains accurate tags on unseen data, and remains practical at scale. Each subsection distinguishes verified evidence from the complete experiment required for a final answer.

Experimental Setup

We use two data classes. The *real agent trajectories* comprise 325 readable local trajectories (198 Codex, 50 Claude, 77 Claude subagent) from a multi-month development project and contain 183,714 system observations. The *public annotated trajectories* comprise 15 families used to evaluate mapping and representation coverage, with 47,590 operations from real agent or human executions. These families cover web agents (WebLINX (Lù, Kasner, and Reddy 2024), Mind2Web (Deng et al. 2023), AgentTrek (Xu et al. 2025), and WebShop (Yao et al. 2022)) and API agents (API-Bank (Li et al. 2023), ToolBench (Qin et al. 2024), and tau-bench (Yao et al. 2025)). They also cover coding agents (SWE-agent (Yang et al. 2024)) and mobile/GUI agents (AndroidCtrl (Li et al. 2024), GUI-Odyssey (Lu et al. 2025), and ScaleCUA (Liu et al. 2026b)). The remaining four families are AgentRewardBench (Lù et al. 2025), SATraj-OS (Chen et al. 2026), OSWorld-Human (Abhyankar, Qi, and Zhang 2026), and AgentNet (Wang et al. 2025).

Each RQ uses controls appropriate to its question while holding visible inputs fixed within each comparison. RQ1 uses the complete 325-trajectory corpus, four declared grouping projections, and 1,000 session-preserving permutations. RQ2 hides real failure, safety, redundancy, and task-boundary annotations from every non-oracle method and compares localization and inspection work. RQ3 fixes its tag vocabulary and tagger before held-out family evaluation. RQ4 measures release-build parsing, field derivation,

caching, folding, output, CPU, peak memory, and storage. Reference annotations are scoring-only unless a view is explicitly marked as an oracle upper bound.

RQ1: Does Semantic Profiling Improve Resource Attribution?

RQ1 asks whether semantic profiling improves attribution of independently recorded resources across runs. The current evidence establishes that every projection conserves the selected measure and that semantic fields can reunite declared recurring work that flat or session-only profiles mix. The complete answer additionally measures independent tool, span, and process lineage.

We run an ablation over semantic grouping fields on 325 real Codex/Claude trajectories (183,714 system observations). We compare four projections (no tags, session only, prompt only, and session plus prompt) and report two measures: (1) mixed-weight percentage, the fraction of system-effect weight in groups containing multiple prompt-tag categories, and (2) residual percentage, the weight in groups whose majority category accounts for less than 90% of the group’s weight.

Lower values indicate stronger separation of the declared categories. Because the same categories define the reference, both measures describe accounting and compression rather than independently verified intent.

The prompt-tag projection separates the declared categories more strongly, whereas the no-tag projection mixes their contexts (Figure 2). For example, *cargo* test appears 2,903 times but the no-tag projection cannot distinguish review, refactor, research, design, and test contexts.

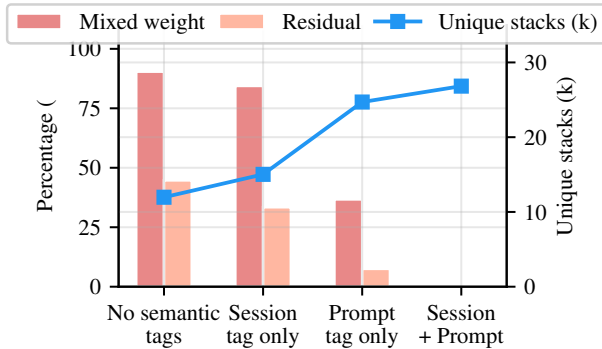


Figure 2: RQ1 grouping ablation over 183,714 observations from 325 trajectories. Bars show mixed-weight and residual percentages, and the line shows unique operation stacks. The selected mixed-weight values are 90.4% without tags, 84.4% with sessions, and 36.7% with prompt tags. Stack counts are 11,967, 15,027, and 24,703. Every projection conserves 183,714 units. This tests declared prompt-tag separation, not lineage.

To test whether prompt tags reduce weighted behavior entropy beyond session membership, let B be the (process, effect, status) tuple, S the session, and P the prompt tag. We report $100[H_w(B | S) - H_w(B | S, P)]/H_w(B |$

$S)$, where H_w is empirical Shannon entropy under system-effect-count weights. In each permutation, we expand integer weights into repeated observations and shuffle P within each fixed S . The observed gain of 8.419% exceeds the null distribution’s 95th percentile of 1.903% over 1,000 permutations ($p = 0.001$). This p-value applies to weighted observations rather than independent sessions and does not establish intent or inheritance correctness.

The operation model supports multiple projections across heterogeneous trajectory sources. Five field selections fold 13,265 operations from nine datasets into 9, 57, 226, 455, or 3,757 groups, and the operation schema represents all 15 families. This coverage makes the same attribution model applicable to agent and human trajectories rather than one source-specific schema.

The selected measure can change which recurring work appears most prominent. Prompt-duration and system-effect-count rankings share seven of their ten highest-ranked categories (Spearman $\rho = 0.623$), with network-tagged prompts ranked 8th and 93rd. The divergence shows why attribution must declare its measure.

RQ1 answer. In the grouping ablation, semantic profiling conserves all 183,714 units and separates recurring declared work that no-tag and session-only profiles mix. The complete attribution evaluation joins real trajectories to independently recorded tool/span identifiers and sandbox process lineage, then measures parent precision/recall, unassigned and duplicate effects, and mass loss against a reference independent of AGENTPROF tags.

RQ2: Does Profiler Output Correspond to Real Problems?

RQ2 tests whether the profiler’s top-ranked groups actually contain real failures, safety violations, or wasted effort. We use public benchmark datasets with independently supplied quality annotations, hide those annotations from the profiler, and check whether its highest-ranked groups recover them. This is analogous to evaluating a CPU profiler against known bottlenecks without giving their locations to the profiler.

The complete experiment covers multiple real agent and human-trajectory families with failure, unsafe-operation, redundant-step, and task-boundary annotations. It compares a flat summary, per-session grouping, the dataset’s native hierarchy, raw-action grouping, and semantic operation-stack profiling. Hidden-annotation views appear only as oracle upper bounds. Every non-oracle method receives the same visible operations and fields.

The primary outcomes are average precision, recall among the highest-ranked groups, recall at a fixed inspection budget, work to the first positive, total inspection work, and group count. The semantic profile supports RQ2 when it concentrates hidden positives into useful groups and reduces analyst inspection relative to flat and per-session views without using the target labels.

RQ2 evaluation. The complete experiment reports all included tasks, all non-oracle baselines, the hidden-label protocol, localization quality, and inspection effort. Intermediate failed constructions remain in experiment history rather than defining the paper’s final result story.

RQ3: How Accurate Are the Tags?

RQ3 asks whether tags remain accurate and stable on unseen agents and task families. A tag is useful only if it is derived without reference annotations, agrees with an independently meaningful task, phase, action, or boundary annotation, and does not materially distort the attribution reported by RQ1.

The complete experiment fixes one tag vocabulary and one target-blind tagger or mapping before target inspection. It leaves out both benchmark families and agent/model families, then compares predicted task, phase, action, and boundary tags with independently supplied annotations using macro-F1, V-measure, boundary precision/recall/F1, abstention, coverage, and stability. Reference annotations never enter field derivation, model selection, or vocabulary revision.

To connect tag accuracy to profiling rather than treat annotation agreement as an end in itself, the experiment replaces predicted tags with independent reference tags and measures changes in RQ1 resource totals and rankings. A clean result must include every held-out family and report contamination checks for mappings, prompts, and ranking choices developed in earlier work.

RQ3 evaluation. The complete target-blind held-out experiment remains required. Historical mapping and boundary scores are omitted until they are revalidated under this protocol.

RQ4: What Is the Profiling Cost?

RQ4 asks whether complete semantic profiling is practical on real trajectory corpora. The relevant cost begins at source parsing and includes uncached field derivation, cache construction, stack folding, serialization, CPU time, peak memory, model or accelerator use, and storage. Repeated queries must be measured separately because their value comes from reusing derived fields while changing the selected measure or operation stack.

The complete experiment uses release builds on naturally occurring full corpora at multiple sizes, including the final RQ1 and RQ2 workloads. It records cold construction, repeated projections from cached fields, scaling with operations and unique tags, and any AgentSight capture overhead when capture is part of the evaluated path. The comparison includes the cost of reviewing raw trajectories so that the result tests whether profiling amortizes the work it replaces.

RQ4 evaluation. The complete experiment reports the scaling matrix, cold and warm costs, resource use, and uncertainty.

Limitations

AGENTPROF is an offline profiler and complements rather than replaces live tracing or single-run debugging. Its operation stacks express accounting ownership, not causation, and their lineage is only as complete as the source fields and recorded intent–effect links. Public benchmark and local development populations may differ from production deployments, so each profile must state its cohort and measure. Our claims therefore concern offline profiles with recorded source fields, declared additive measures, and explicitly evaluated

tags. They do not imply causal ownership or universal behavior across unmeasured deployment populations.

Related Work

Agent tracing and observability. Tracing tools record request traces (LangChain 2024; Langfuse 2024; Arize AI 2024a), and OpenTelemetry/OpenInference standardize GenAI spans (OpenTelemetry 2024; Arize AI 2024b). AgentOps and provenance work organize retained evidence, while AgentTelemetry benchmarks telemetry-based fault detection (Dong, Lu, and Zhu 2024; Wang et al. 2026b; Balusu 2026). Datadog and Laminar derive topics or signals (Datadog 2025; Laminar 2025), not recurring-measure attribution.

Profiling and aggregation. Domain-specific profiling builds hierarchies along selected dimensions (Sloane and Roberts 2014). Perfetto/pprof compute prefix aggregates (Google 2024a,b), Pivot Tracing groups causally related metrics (Mace, Roelke, and Fonseca 2015), and aggregate-trace work summarizes similar traces (Samanta et al. 2024). AGENTPROF tests attribution projections for recurring agent activity.

Failure localization and diagnostic benchmarks. AgentRx/TELBench localize steps or spans, TrajAD marks anomalies, and AgentFixer adds root-cause and repair recommendations (Barke et al. 2026; Wang et al. 2026a; Liu et al. 2026a; Mulian et al. 2026). AgentAtlas supplies a diagnostic vocabulary and audit protocol (Mazaheri and Mazaheri 2026). TS-Bench, RedundancyBench, and SDBL contribute safety, redundancy, and failure-scope labels (Mou et al. 2026; Hu et al. 2026; Sun et al. 2026), while Who&When and TRAIL add agent/time responsibility and error taxonomies (Zhang et al. 2025; Deshpande et al. 2025). These systems provide independently grounded problems, labels, and strong baselines for testing whether a profile corresponds to real agent behavior.

Semantic trajectory analysis. Hodoscope compares semantic behavior distributions across agent cohorts (Zhong, Saxena, and Raghunathan 2026); AgentDiagnose visualizes trajectory behavior, and ARIA aggregates rewards over semantically similar actions (Ou et al. 2025; Yang et al. 2025). These systems establish semantic cross-run analysis as an important neighboring problem. AGENTPROF instead organizes recorded resource and effect measures as profiles and evaluates attribution, real-problem localization, tag accuracy, and profiling cost.

Conclusion

Agent observability needs profiling, not only debugging. AGENTPROF shows how a semantic operation stack model with pluggable intent attribution and stack construction brings hierarchical resource attribution to agent trajectories. Uniform operations connect agent intent, model and tool activity, and recorded system effects; query-time operation stacks fold this activity by task, phase, action, or another semantic dimension. The four-part evaluation asks whether semantic profiling improves resource attribution, locates real annotated problems, derives accurate tags, and remains practical. Profiling complements per-run debugging

by turning many agent trajectories into aggregate views of quality, safety, and cost.

References

- Abhyankar, R.; Qi, Q.; and Zhang, Y. 2026. OSWorld-Human: Benchmarking the Efficiency of Computer-Use Agents. In *Proceedings of the 9th Annual Conference on Machine Learning and Systems (MLSys)*.
- Anthropic. 2025. Claude Code: An Agentic Coding Tool. <https://code.claude.com/docs/en/overview>.
- Arize AI. 2024a. Arize Phoenix. <https://arize.com/docs/phoenix>.
- Arize AI. 2024b. OpenInference Specification. <https://arize-ai.github.io/openinference/spec/>.
- Balusu, K. C. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26)*.
- Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.
- Chen, X.; Yin, Z.; He, S.; Huang, B.; Lei, S.; et al. 2026. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230.
- Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.
- Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
- Deshpande, D.; Gangal, V.; Mehta, H.; Krishnan, J.; Kannappan, A.; and Qian, R. 2025. TRAIL: Trace Reasoning and Agentic Issue Localization. arXiv preprint arXiv:2505.08638.
- Dong, L.; Lu, Q.; and Zhu, L. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285.
- Google. 2024a. Perfetto Trace Processor. <https://perfetto.dev/docs/analysis/trace-processor>.
- Google. 2024b. pprof. <https://github.com/google/pprof>.
- Gregg, B. 2011. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- Hu, M.; Yang, B.; Zhou, Z.; Liang, J.; Guo, J.; Yin, Y.; and Han, X. 2026. Redundant or Necessary? A Benchmark for Detecting Redundant Steps in Agent Trajectories. arXiv preprint arXiv:2605.29893.
- Laminar. 2025. Signals: Structured Event Extraction from Traces. <https://docs.lmn.ai/signals/introduction>.
- LangChain. 2024. LangSmith Observability. <https://docs.langchain.com/langsmith/observability>.
- Langfuse. 2024. Langfuse Observability. <https://langfuse.com/docs/observability/overview>.
- Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 3102–3116.
- Li, W.; Bishop, W. E.; Li, A.; Rawles, C.; Campbell-Ajala, F.; Tyamagundlu, D.; and Riva, O. 2024. On the Effects of Data Scale on UI Control Agents. In *Advances in Neural Information Processing Systems*, 92130–92154. Datasets and Benchmarks Track.
- Liu, Y.; Zhang, C.; Han, Z.; Liu, H.; Wang, Y.; Yu, Y.; Wang, X.; and Yin, Y. 2026a. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. arXiv preprint arXiv:2602.06443.
- Liu, Z.; Xie, J.; Ding, Z.; Li, Z.; Yang, B.; et al. 2026b. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. In *ICLR 2026*.
- Lu, Q.; Shao, W.; Liu, Z.; Du, L.; Meng, F.; Li, B.; Chen, B.; Huang, S.; Zhang, K.; and Luo, P. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 22404–22414.
- Lù, X. H.; Kasner, Z.; and Reddy, S. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. In *ICML*, volume 235 of *Proceedings of Machine Learning Research*, 33007–33056.
- Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. In *Proceedings of the Second Conference on Language Modeling (COLM)*.
- Mace, J.; Roelke, R.; and Fonseca, R. 2015. Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems. In *Proceedings of the 25th Symposium on Operating Systems Principles*, 378–393.
- Mazaheri, P.; and Mazaheri, K. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530.
- Mou, Y.; Xue, Z.; Li, L.; Liu, P.; Zhang, S.; Ye, W.; and Shao, J. 2026. ToolSafe: Enhancing Tool Invocation Safety of LLM-based Agents via Proactive Step-level Guardrail and Feedback. In *Findings of the Association for Computational Linguistics: ACL 2026*, 37125–37153.
- Mulian, H.; Zeltyn, S.; Levy, I.; Galanti, L.; Yaeli, A.; and Shlomov, S. 2026. AgentFixer: From Failure Detection to Fix Recommendations in LLM Agentic Systems. arXiv preprint arXiv:2603.29848; presented at AGENT 2026.
- OpenAI. 2025. Codex CLI. <https://github.com/openai/codex>.
- OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. <https://github.com/open-telemetry/semantic-conventions-genai>.
- Ou, T.; Guo, W.; Gandhi, A.; Neubig, G.; and Yue, X. 2025. AgentDiagnose: An Open Toolkit for Diagnosing

- LLM Agent Trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 207–215. Association for Computational Linguistics.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerstein, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- Samanta, A.; Han, H.; Huye, D.; Liu, L.; Zhang, Z.; and Sambasivan, R. R. 2024. Visualizing Distributed Traces in Aggregate. arXiv preprint arXiv:2412.07036.
- Sloane, A. M.; and Roberts, M. 2014. Domain-Specific Program Profiling and Its Application to Attribute Grammars and Term Rewriting. *Science of Computer Programming*, 96(P4): 488–510.
- Sun, K.; Li, W.; Dong, B.; Lin, Y.; Zhang, J.; and Shi, B. 2026. Scope Delineation Before Localization: A Two-Stage Framework for Enhancing Failure Attribution in Multi-Agent Systems. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(39): 33108–33116.
- Wang, J.; Feng, Z.; Wu, J.; Li, R.; Xie, Q.; Ren, Y.; Zhu, H.; Han, X.; Meng, F.; Feng, J.; and Liu, J. 2026a. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060.
- Wang, X.; Wang, B.; Lu, D.; Yang, J.; Xie, T.; et al. 2025. OpenCUA: Open Foundations for Computer-Use Agents. In *NeurIPS 2025*.
- Wang, Y.; Zhang, J.; Cai, T.; Liu, Z.; Sun, Q.; Sun, Z.; Wu, Z.; Dong, M.; Zheng, M.; Yin, X.; and Zhu, Y. 2026b. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv preprint arXiv:2606.04990.
- Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OS-World: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems. Datasets and Benchmarks Track*.
- Xu, Y.; Lu, D.; Shen, Z.; Wang, J.; Wang, Z.; Mao, Y.; Xiong, C.; and Yu, T. 2025. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. In *ICLR 2025*.
- Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.
- Yang, R.; Zhang, Y.; Chen, A.; Wang, X.; Yuan, S.; Chen, J.; Yang, D.; and Xiao, Y. 2025. ARIA: Training Language Agents with Intention-Driven Reward Aggregation. In *NeurIPS 2025*.
- Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. Web-Shop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.
- Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *ICLR 2025*.
- Zhang, S.; Yin, M.; Zhang, J.; Liu, J.; Han, Z.; Zhang, J.; Li, B.; Wang, C.; Wang, H.; Chen, Y.; and Wu, Q. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 76583–76599.
- Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
- Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems (PACML '25, co-located with SOSPL)*.
- Zhong, Z.; Saxena, S.; and Raghunathan, A. 2026. Hodoscope: Unsupervised Monitoring for AI Misbehaviors. arXiv preprint arXiv:2604.11072.